

Laboratorio Avanzado de Patrones NoSQL: Cassandra

Departamento de Informática

22 de abril de 2026

1 Introducción al Laboratorio

En este laboratorio exploraremos los patrones de diseño que hacen que gigantes como Netflix o Instagram usen Cassandra. No se trata solo de guardar datos, sino de modelarlos para que la lectura sea instantánea independientemente del volumen.

1.1 Patrón 1: Series Temporales (Telemetría IoT)

Escenario: Queremos guardar la temperatura de los sensores de los hoteles de GloboTour cada minuto.

Diseño de la Tabla

```
CREATE TABLE sensor_readings (  
  hotel_id uuid,  
  sensor_id text,  
  day date,  
  reading_time timestamp,  
  value float,  
  PRIMARY KEY ((hotel_id, day), reading_time)  
) WITH CLUSTERING ORDER BY (reading_time DESC);
```

Análisis del Patrón:

- **Composite Partition Key (hotel_id, day):** Agrupamos las lecturas por hotel y por día. Esto evita que una partición crezca demasiado con el paso de los años, manteniendo el rendimiento constante.
 - **Uso Práctico:** Permite responder a «Dame todas las temperaturas de hoy del Hotel X» de forma ultra-eficiente.
-

1.2 Patrón 2: Contadores Atómicos (Votos y Likes)

Escenario: Necesitamos contar cuántas veces se ha visualizado cada oferta de viaje sin que las escrituras masivas bloqueen la tabla.

Uso de la tabla `offer_stats`

```
-- Nota: Las tablas de contadores solo pueden tener contadores y la PK.  
UPDATE offer_stats  
SET views = views + 1  
WHERE offer_id = 'OFERTA-PARIS-2024';
```

Análisis del Patrón: Cassandra gestiona los contadores de forma especial para evitar «Race Conditions» sin necesidad de bloqueos de fila de SQL.

1.3 Patrón 3: Datos con Caducidad (TTL)

Escenario: Marketing lanza ofertas «Flash» que solo duran 2 horas. No queremos un proceso que borre datos viejos (que es lento); queremos que los datos **mueran solos**.

Inserción con TTL

```
INSERT INTO flash_offers (offer_id, discount)  
VALUES ('REBAJA-10', 0.15)  
USING TTL 7200; -- Expira en 2 horas (7200 seg)
```

1.4 Laboratorio de Depuración: ¿Por qué falla mi CQL?

- **Error de Filtrado:** `SELECT * FROM sensor_readings WHERE value > 25;` → **FALLARÁ**. Cassandra no sabe en qué nodo está ese valor sin la clave de partición.
- **Error de Ordenación:** `SELECT * FROM sensor_readings ORDER BY value DESC;` → **FALLARÁ**. Solo puedes ordenar por las *Clustering Columns* definidas en el esquema.

La Regla de Oro

En Cassandra, si no puedes responder a una pregunta con tu tabla actual, **crea otra tabla**. El disco es barato, el tiempo de tus usuarios no.